Project Blueprint: GraphRAG Research Engine

Architecting the Future of Contextual AI

1. Project Vision & Core Problem

Standard AI (RAG) looks for keywords but fails to understand the **relationships** between ideas. **GraphRAG** solves this by mapping data as a "Knowledge Graph." If you upload a research paper, the AI doesn't just see sentences; it sees how one variable impacts another through structured connections.

2. Technical Architecture

Next.js 14 | FastAPI (Python) | Neo4j (Graph DB) | Pinecone (Vector) | LangChain

Layer	Technology	Responsibility
Frontend	Next.js + Tailwind	Interactive Graph UI & Chat Interface
API Layer	FastAPI	Orchestrating AI agents and Database queries
Knowledge Layer	Neo4j	Storing entities and their complex relationships

Semantic Layer	Pinecone/ Chroma	Similarity search for quick text retrieval

3. Key Concepts to Master

A. Knowledge Graphs (The "Brain")

Instead of a flat table, data is stored as **Nodes** (Entities) and **Edges** (Relationships). This allows the AI to perform "multi-hop" reasoning—finding answers that aren't in a single sentence but spread across a document.

B. Retrieval-Augmented Generation (RAG)

Giving an LLM "open-book" access to your data. We use **Vector Embeddings** to find relevant text chunks and **Cypher Queries** to find relevant graph connections.

4. Implementation Roadmap

Phase 1: Setup - VS Code environment, Python virtual environments, and API keys.

Phase 2: Ingestion - PDF parsing and Entity Extraction (using AI to find 'Names' and 'Connections').

Phase 3: Logic - Building the FastAPI backend to query Neo4j.

Phase 4: UI - A clean, high-end dashboard to visualize the graph and chat.

Phase 5: Deploy - Pushing to GitHub and deploying via Railway and Vercel.